Name: Shravani Kurkute
PRN: 22311926
Roll No.: 421129 **Div:** Btech-A **Subject:** NLP

Assignment No. 4

Code:

> BuildDatasets.py

```
import argparse
import csv
import os
import random

from gensim.models import KeyedVectors


CAPITAL_COUNTRY_DATA = """athens greece|baghdad iraq|bangkok thailand|beijing
china|berlin germany|bern switzerland|cairo egypt|
canberra australia|copenhagen denmark|dublin ireland|hanoi vietnam|helsinki finland|islamabad
pakistan|jakarta indonesia|
kabul afghanistan|lima peru|lisbon portugal|madrid spain|manila philippines|moscow
russia|nairobi kenya|oslo norway|
ottawa canada|paris france|rome italy|stockholm sweden|tehran iran|tokyo japan|vienna
austria|warsaw poland|havana cuba|
budapest hungary|brussels belgium|ankara turkey|amman jordan|beirut lebanon|damascus
syria|kiev ukraine|santiago chile|
bogota colombia|caracas venezuela|quito ecuador|montevideo uruguay|tripoli libya|tunis
tunisia|algiers algeria|rabat morocco|
khartoum sudan|accra ghana|abuja nigeria|kampala uganda|harare zimbabwe|lusaka
zambia|mogadishu somalia|kathmandu nepal|
dhaka bangladesh|tashkent uzbekistan|baku azerbaijan|tbilisi georgia|yerevan armenia|minsk
belarus|vilnius lithuania|
riga latvia|tallinn estonia|sofia bulgaria|bucharest romania|belgrade serbia|zagreb croatia|ljubljana
slovenia|
bratislava slovakia|reykjavik iceland|valletta malta|nicosia cyprus|doha qatar|muscat
oman|vientiane laos|delhi india|
london england""".replace("\n", "").split("|")
ADDITIONAL_COUNTRIES = """brazil argentina mexico israel ethiopia tanzania mozambique
angola cameroon senegal mali niger chad
bolivia paraguay panama honduras guatemala nicaragua jamaica haiti kuwait bahrain yemen
turkmenistan kazakhstan kyrgyzstan
tajikistan albania macedonia bosnia montenegro moldova luxembourg liechtenstein monaco
singapore malaysia myanmar cambodia
```

```
bhutan fiji tonga samoa madagascar botswana namibia rwanda burundi malawi liberia eritrea
djibouti""".split()


def load_embeddings(model_path=None):
 print("[1/4] Loading GloVe embeddings...", flush=True)

 if model_path and model_path.endswith(".kv"):
 embeddings = KeyedVectors.load(model_path)

 elif model_path:
 embeddings = KeyedVectors.load_word2vec_format(
 model_path,
 binary=False,
 no_header=True
 )

 else:
 import gensim.downloader as api

 print(
 "Downloading/loading glove-wiki-gigaword-100. "
 "The first run may take time.",
 flush=True
 )
 embeddings = api.load("glove-wiki-gigaword-100")

 print(
 f"Model loaded: {len(embeddings.key_to_index):,} words.",
 flush=True
 )

 return embeddings
def generate_analogies(valid_pairs, count, seed):
available = len(valid_pairs) * (len(valid_pairs) - 1)

 if count > available:
 raise ValueError(
 f"Cannot generate {count} unique analogies from "
f"{len(valid_pairs)} valid pairs. "
 f"Maximum possible: {available}."
 )

 rng = random.Random(seed)
```

```
 used_records = set()
 records = []

 while len(records) < count:
 (capital1, country1), (capital2, country2) = rng.sample(
valid_pairs, 2
 )

 record = (capital1, country1, capital2, country2)

 if record in used_records:
 continue

 used_records.add(record)
 records.append(record)

 return records


def save_csv(file_path, headers, records):
 with open(file_path, "w", newline="") as file:
writer = csv.writer(file)
 writer.writerow(headers)
 writer.writerows(records)


def main():
 parser = argparse.ArgumentParser()
 parser.add_argument(
 "--model-path",
 default=None,
 help="Path to a local GloVe text/gzip file or a .kv file"  )

 parser.add_argument(
 "--out",
 default=os.path.join(
 os.path.dirname(os.path.abspath(__file__)),  "..",
 "datasets"
 )
 )

 args = parser.parse_args()
 os.makedirs(args.out, exist_ok=True)
```

```
 embeddings = load_embeddings(args.model_path)

print("[2/4] Preparing capital-country pairs...", flush=True)

 valid_pairs = []
 seen_pairs = set()

 for entry in CAPITAL_COUNTRY_DATA:
capital, country = entry.split()

 if (capital, country) in seen_pairs:
 continue

 seen_pairs.add((capital, country))

 if (
 capital in embeddings.key_to_index
 and country in embeddings.key_to_index  ):
 valid_pairs.append((capital, country))
 else:
 print("Skipped (not in vocabulary):", capital, country)

print(f"Valid pairs: {len(valid_pairs)}", flush=True)

 save_csv(
 os.path.join(args.out, "capitals_countries.csv"),
 ["capital", "country"],
 valid_pairs
 )

 candidates = sorted(
 {country for _, country in valid_pairs}
 | {
 country for country in ADDITIONAL_COUNTRIES  if
country in embeddings.key_to_index
 }
 )

 save_csv(
 os.path.join(args.out, "candidate_countries.csv"),
["country"],
 [[country] for country in candidates]
 )
```

```
 print(
 f"Candidate countries: {len(candidates)}",
 flush=True
 )

 print("[3/4] Generating test and evaluation datasets...", flush=True)

example = ("athens", "greece", "baghdad", "iraq")

 test_records = [example] + [
 record
 for record in generate_analogies(valid_pairs, 40, 42)  if
record != example
 and record[1] != "greece"
 and record[3] != "iraq"
 ][:14]
 large_records = generate_analogies(valid_pairs, 1000, 7)

 save_csv(
 os.path.join(args.out, "test_analogies.csv"),
["capital1", "country1", "capital2", "country2"],
test_records
 )

 save_csv(
 os.path.join(args.out, "eval_analogies_large.csv"),
["capital1", "country1", "capital2", "country2"],
large_records
 )

 print("[4/4] Dataset generation completed.", flush=True)
print(f"Test analogies: {len(test_records)}")
print(f"Evaluation analogies: {len(large_records)}")
print(f"Output directory: {os.path.abspath(args.out)}")


if __name__ == "__main__":
 main()
```

> 421006_Tushar_Neje_NLP_Assignment04.py

```
import argparse
import csv
import json
import os
import time

import numpy as np
 from gensim.models import KeyedVectors


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "..", "datasets")
RESULTS_DIR = os.path.join(BASE_DIR, "..", "results")

BATCH_SIZE = 16


def load_embeddings(model_path=None):
 start_time = time.time()

 print("[model] Loading embeddings...", flush=True)

 if model_path is None:
 import gensim.downloader as api
 embeddings = api.load("glove-wiki-gigaword-100")

 elif model_path.endswith(".kv"):
 embeddings = KeyedVectors.load(model_path)

 else:
 embeddings = KeyedVectors.load_word2vec_format(
model_path,
 binary=False
 )

 print(
 f"[model] Loaded {len(embeddings.key_to_index):,} words, "
f"dimension {embeddings.vector_size}, "
 f"in {time.time() - start_time:.1f}s",
 flush=True
 )

 return embeddings
```

```
def load_records(filename):
 file_path = os.path.join(DATA_DIR, filename)

 with open(file_path, newline="") as file:
 return list(csv.DictReader(file))
class CountryAnalogyPredictor:
 def __init__(self, embeddings, candidate_countries):
self.embeddings = embeddings
 self.normalized_vectors = embeddings.get_normed_vectors()
self.vocabulary = embeddings.index_to_key

 self.countries = [
 country
 for country in candidate_countries
 if country in embeddings.key_to_index
 ]

 self.country_indices = np.array([
 embeddings.key_to_index[country]
 for country in self.countries
 ])

 self.country_vectors = self.normalized_vectors[
self.country_indices
 ]

 def build_analogy_vector(self, capital1, country1, capital2):
get_vector = lambda word: self.normalized_vectors[
self.embeddings.key_to_index[word]
 ]

 return (
 get_vector(country1)
 - get_vector(capital1)
 + get_vector(capital2)
 )

 @staticmethod
 def normalize_vector(vector):
 return vector / np.linalg.norm(vector)

 def rank_candidates_batch(
```

```
 self,
 records,
 topn=5,
 country_filter=True,
 batch_size=BATCH_SIZE
 ):
 all_results = []
 total_records = len(records)
 show_progress = total_records >= 100

 for start in range(0, total_records, batch_size):
batch = records[start:start + batch_size]

 query_vectors = []
 excluded_words = []

 for record in batch:
 capital1, country1, capital2 = (
 record["capital1"].strip().lower(),
 record["country1"].strip().lower(),
 record["capital2"].strip().lower()
 )

 for word in (capital1, country1, capital2):  if word not in
self.embeddings.key_to_index:  raise KeyError(
 f"'{word}' is not in the embedding vocabulary"  )

 analogy = self.build_analogy_vector(
 capital1,
 country1,
 capital2
 )

 query_vectors.append(
 self.normalize_vector(analogy)
 )

 excluded_words.append({
 capital1,
 country1,
 capital2
 })

 query_matrix = np.asarray(
```

```
 query_vectors,
 dtype=self.normalized_vectors.dtype  )

 # One matrix multiplication processes the entire batch.  if
country_filter:
 similarity_matrix = (
 query_matrix @ self.country_vectors.T  )
 else:
 similarity_matrix = (
 query_matrix @ self.normalized_vectors.T  )

 for row_index, scores in enumerate(similarity_matrix):  if
country_filter:
 indices = np.argsort(-scores)

 ranked_words = [
 (
 self.countries[index],
 float(scores[index])
 )
 for index in indices
 ]

 else:
 # Preserve the original top-200 vocabulary search.  indices =
np.argpartition(
 -scores, 200
 )[:200]

 indices = indices[
 np.argsort(-scores[indices])
 ]

 ranked_words = [
 (
 self.vocabulary[index],
 float(scores[index])
 )
 for index in indices
 if self.vocabulary[index].isalpha()
 ]

 excluded = excluded_words[row_index]
```

```
ranked_results = [
(word, score)
for word, score in ranked_words
if word not in excluded
][:topn]

all_results.append(ranked_results)

completed = start + len(batch)

if show_progress and (
completed // 100 > start // 100
or completed == total_records
):
print(
f"\r[ranking] {completed}/{total_records} analogies",  end="",
flush=True
)

if show_progress:
print(flush=True)

return all_results

def rank_candidates(
self,
capital1,
country1,
capital2,
topn=5,
country_filter=True
):
record = {
"capital1": capital1,
"country1": country1,
"capital2": capital2
}

return self.rank_candidates_batch(
[record],
topn=topn,
country_filter=country_filter,
batch_size=1
```

```
 )[0]

 def predict_country(
 self,
 capital1,
 country1,
 capital2,
 country_filter=True
 ):
 ranked = self.rank_candidates(
capital1,
 country1,
 capital2,
 topn=1,
 country_filter=country_filter  )

 return ranked[0][0]


 def evaluate_predictions(
 predictor,
 records,
 country_filter=True,
 topk=5
):
 results = []
 top1_correct = 0
 topk_correct = 0

 ranked_batches = predictor.rank_candidates_batch(
records,
 topn=topk,
 country_filter=country_filter,
 batch_size=BATCH_SIZE
 )

 for record, ranked in zip(records, ranked_batches):
predicted_country = ranked[0][0]
 is_correct = predicted_country == record["country2"]

top1_correct += is_correct

 topk_correct += (
 record["country2"]
```

```
 in [word for word, _ in ranked]
 )

 results.append({
 **record,
 "predicted": predicted_country,
 "correct": is_correct,
 "top5": ranked
 })

 return results, top1_correct, topk_correct, len(records)


def display_example(
 predictor,
 capital1,
 country1,
 capital2,
 actual=None
):
 for filter_countries in (True, False):
 ranked = predictor.rank_candidates(
 capital1,
 country1,
 capital2,
 topn=5,
 country_filter=filter_countries
 )

 label = (
 "country-filtered"
 if filter_countries
 else "full vocabulary "
 )

 formatted_results = ", ".join(
 f"{word} ({score:.3f})"
 for word, score in ranked
 )

 print(f" [{label}] {formatted_results}")

 # Reuse the country-filtered result instead of ranking a third time.
filtered_results = predictor.rank_candidates(
```

```
 capital1,
 country1,
 capital2,
 topn=1,
 country_filter=True
 )

 prediction = filtered_results[0][0]

 if actual is None:
 status = ""
 elif prediction == actual:
 status = " -> CORRECT"
 else:
 status = f" -> WRONG (expected {actual})"

 print(
 f" Predicted Country 2 = "
 f"{prediction.upper()}{status}"
 )


def save_predictions(file_path,
predictions):  headers = [
 "capital1",
 "country1",
 "capital2",
 "actual_country2",
 "predicted",
 "correct",
 "cand1",
 "score1",
 "cand2",
 "score2",
 "cand3",
 "score3"
 ]

 with open(file_path, "w", newline="") as output_file:
writer = csv.writer(output_file)
 writer.writerow(headers)

 for record in predictions:
 row = [
```

```
 record["capital1"],
 record["country1"],
 record["capital2"],
 record["country2"],
 record["predicted"],
 "Correct" if record["correct"] else "Incorrect"  ]

 for word, score in record["top5"][:3]:
row.extend([word, f"{score:.4f}"])

 writer.writerow(row)
def run_evaluation(predictor):
 os.makedirs(RESULTS_DIR, exist_ok=True)

 datasets = [
 ("test15", load_records("test_analogies.csv")),  ("large1000",
load_records("eval_analogies_large.csv"))  ]

 summary = {}

 for dataset_name, records in datasets:
 print(
 f"\n[evaluation] Dataset: {dataset_name} "
f"({len(records)} analogies)",
 flush=True
 )

 for country_filter in (True, False):
 filter_name = (
 "country_filtered"
 if country_filter
 else "full_vocab"
 )

 print(
 f"[evaluation] Mode: {filter_name}",
flush=True
 )

 predictions, top1, top5, total = evaluate_predictions(
predictor,
 records,
 country_filter=country_filter,
 topk=5
```

```
 )

 result_key = f"{dataset_name}_{filter_name}"

summary[result_key] = {

 "total": total,
 "top1_correct": int(top1),
 "top1_acc_pct": round(100 * top1 / total, 2),
"top5_correct": int(top5),
 "top5_acc_pct": round(100 * top5 / total, 2)  }

 output_path = os.path.join(
 RESULTS_DIR,
 f"predictions_{result_key}.csv"
 )

 save_predictions(output_path, predictions)

 summary_path = os.path.join(
 RESULTS_DIR,
 "accuracy_summary.json"
 )

 with open(summary_path, "w") as summary_file:
json.dump(summary, summary_file, indent=2)

 print("\n" + "=" * 78)
 print("ACCURACY SUMMARY")
 print("=" * 78)

 for name, metrics in summary.items():
 print(
 f"{name:<28} "
 f"n={metrics['total']:<5} "
 f"top-1: {metrics['top1_correct']:>4} "
 f"({metrics['top1_acc_pct']:>6.2f}%) "
 f"top-5: {metrics['top5_acc_pct']:>6.2f}%"  )

 print(f"\nResults saved to: {os.path.abspath(RESULTS_DIR)}")


def main():
 parser = argparse.ArgumentParser(
```

```
 description="GloVe-based country analogy predictor",
formatter_class=argparse.RawDescriptionHelpFormatter  )

 parser.add_argument("--model-path", default=None)

 parser.add_argument(
 "--words",
 nargs=3,
 metavar=("CAPITAL1", "COUNTRY1", "CAPITAL2")  )

 parser.add_argument("--demo", action="store_true")
parser.add_argument("--evaluate", action="store_true")

 args = parser.parse_args()

 embeddings = load_embeddings(args.model_path)

 candidate_countries = [
 record["country"]
 for record in load_records("candidate_countries.csv")  ]

 predictor = CountryAnalogyPredictor(
 embeddings,
 candidate_countries
 )

 print(
 f"[data] {len(predictor.countries)} "
 f"candidate countries loaded\n",
 flush=True
 )

 if args.words:
 print(
 f"Input: {args.words[0]} - "
 f"{args.words[1]} + {args.words[2]}"
 )
 display_example(predictor, *args.words)
 return

 if args.demo or args.evaluate:
 test_records = load_records("test_analogies.csv")

 print("=" * 78)
```

```
 print("EXAMPLE FROM THE ASSIGNMENT SHEET")
print("=" * 78)
 print("Athens : Greece :: Baghdad : ?")

 display_example(
 predictor,
 "athens",
 "greece",
 "baghdad",
 "iraq"
 )

 print("\n" + "=" * 78)
 print(f"TEST SET ({len(test_records)} analogies)")
print("=" * 78)

 for index, record in enumerate(test_records, 1):
print(
 f"{index:>2}. "
 f"{record['capital1'].title()} : "
 f"{record['country1'].title()} :: "
 f"{record['capital2'].title()} : ?"
 )

 display_example(
 predictor,
 record["capital1"],
 record["country1"],
 record["capital2"],
 record["country2"]
 )
 if args.evaluate:
 run_evaluation(predictor)
 return

 if args.demo:
 return

 print(
 "Interactive mode - type three words "
"(Capital1 Country1 Capital2), or 'q' to quit."  )

 while True:
 user_input = input("\n> ").strip().lower().split()
```

```
 if user_input and user_input[0] in (
"q", "quit", "exit"
 ):
 break

 if len(user_input) != 3:
 print(
 "Please enter exactly three words, "  "e.g.:
athens greece baghdad"  )
 continue

 try:
 display_example(predictor, *user_input)
except KeyError as error:
 print("Error:", error)


if __name__ == "__main__":
 try:
 from threadpoolctl import threadpool_limits
except ImportError:
 print(
 "threadpoolctl is not installed. "  "Install it
with: pip install threadpoolctl",
 flush=True
 )
 main()
 else:
 cpu_cores = os.cpu_count() or 1

 print(
 f"[cpu] Configuring BLAS to use up to "
 f"{cpu_cores} CPU threads",
 flush=True
 )

 with threadpool_limits(
 limits=cpu_cores,
 user_api="blas"
 ):
 main()
```

Execution:

Output: